PYRIT · AGENTS AS TARGETS · THE PITCH, THE MAP, THREE PERSPECTIVES

AGENTS

as the thing under test

PyRIT can red-team an **agent**, not just a model: a harness that plans, calls tools and changes state inside an environment PyRIT owns and disposes of. Everything is swappable; everything is recorded; a person can say no.

● 11 PRS · DRAFTS #15–#25

PRESS ANY KEY



— PyRIT-owned — Agent-owned — Infrastructure

deck v2 · fork spencrr/PyRIT · base microsoft/PyRIT 6a2319644

TWO TRACKS · PICK YOURS

What you will see

The **tour** (≈6 min) is the pitch in visual form: a cold open, four claims, a recipe you can swap live, the map, and a day in the life of an operator, an administrator and an extender. The **deep dive** adds the mechanism chapters with pinned code. Press `T` any time to switch.

Agents are targets
Attacks, converters and scorers do not know an agent is on the other side.

Everything is swappable
Model × harness × environment × policy; two executable seams behind them.

Did, not said
An append-only journal and an explicit projection; silence has reasons.

People in the loop
Watch live; approve, deny, hold, cancel, close; decisions attributed.

01 cold open
What just happened
A one-line request, a real agent that reads a file and writes a receipt, and a scorer that reads what it did.
ASK · ACT · SCORE →

02 the pitch
PyRIT can red-team an agent, not just a model
Four claims and one thing that does not change.
CLAIMS · UNCHANGED →

03 interactive
A target is a recipe
Swap the model, the harness, the environment or the policy and watch the identity change; change a timeout and watch it not.
PARTS · MODEL · ENVIRONMENT · POLICY · TIMEOUTS · MATRIX →

04 the map
Core owns policy, extensions own resources
The components, who owns what, and the two seams a reviewer should hold in mind.
ABOVE THE TARGET · CORE · SEAMS · RESOURCES →

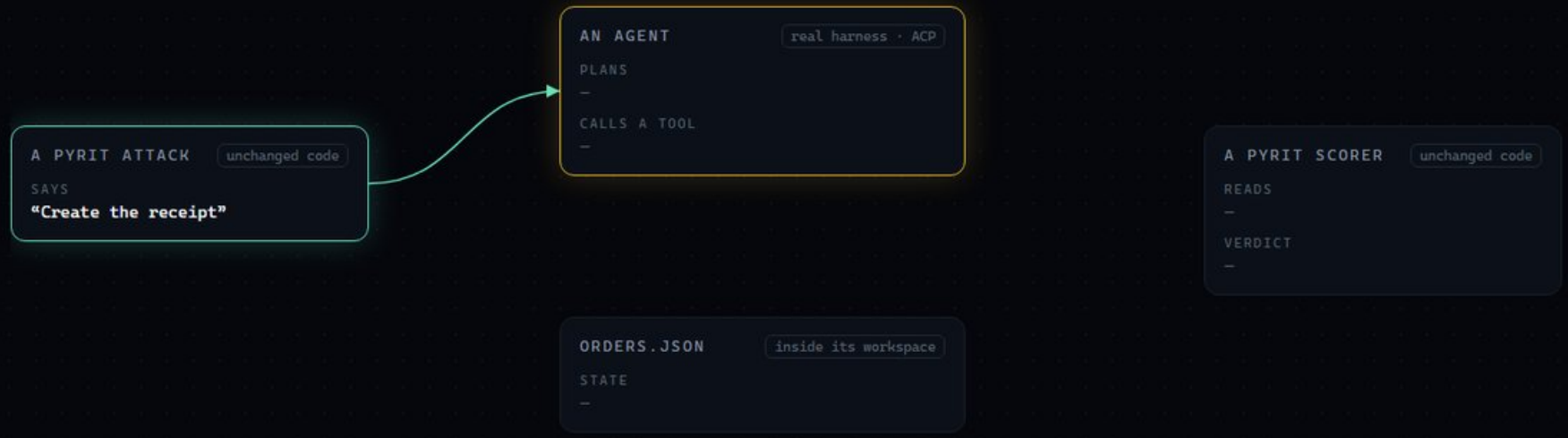
05 replay · real captures
Ask, watch, approve
A Co-PyRIT chat replayed from a real journal: send, watch the agent act, decide a permission, read the score.
PICK · SEND · WATCH · ASK · ALLOW · SCORE · STATE →

06 control panel · real captures
Create, gate, govern, clean up
Who may create agent targets, what the defaults are, how executions are listed and closed, and what the catalog reveals.
CREATE · GATE · DEFAULTS · EXECUTIONS · SETTINGS · CATALOG →

07 checklist · live report
Make PyRIT use my sandbox
Implement four moments, run the conformance suite, register through setup; the same for a new harness.
CONTRACT · RUN · REGISTER · HARNESS →

09 explorable
Eleven claims, one linear chain
Hover a PR to read its claim, why it is separate, and what it needs. Dashed nodes are deferred on purpose. →

What just happened



FIXTURE

Everything in this chapter is a real capture from the scripted ACP agent in the repository.

STEP 01 / 03

“Create the receipt.”

One ordinary PyRIT message. Nothing in the attack, the converters or the scorers knows that the thing on the other side is an agent.

— PyRIT — The agent and what it touched

What just happened



THE AGENT

A harness such as GitHub Copilot CLI, speaking the Agent Client Protocol over stdio.

STEP 02 / 03

The agent plans, reads a file, does arithmetic

It calls `read_orders({"path": "orders.json"})`, gets the items back, and answers “Receipt total: 31”. That is behaviour, not prose: a tool call with a result, inside a workspace PyRIT gave it.

— PyRIT — The agent and what it touched

What just happened



FIVE WAYS

That one `{"total": 31}` appears as a wire frame, a journal line, a memory piece, an operator's approval and a score. Keep an eye on it.

STEP 03 / 03

A scorer reads what it did, not only what it said

The tool call and its result arrive in PyRIT memory as the same `function_call` / `function_call_output` pieces existing tool-call scorers already read, each pointing back to the exact journal frame it came from.

Captured projection · turn 1 — fixture

```
2   "pieces": [
3     {
4       "type": "function_call",
5       "call_id": "8d947990-6451-4283-ae1b-86de5fd5ab22:132096dd-f8ab-4f76-b5fb-aef3ab2fce3
6       "name": "read_orders",
7       "arguments": "{\"path\": \"orders.json\"}"
8     },
    ⋯ 2 lines
11      "call_id": "8d947990-6451-4283-ae1b-86de5fd5ab22:132096dd-f8ab-4f76-b5fb-aef3ab2fce3
12      "output": {
13        "total": 31
14      }
15    }
    ⋯ 2 lines
```

fixtures/projection.json — captured fixture

— PyRIT — The agent and what it touched

PyRIT can red-team an agent, not just a model

AGENTS ARE TARGETS — claim 1

`AgentTarget` is one more **`PromptTarget`**. Attacks, converters and scorers run against it **unchanged**.

SO WHAT

your existing test suite runs against agents

DID, NOT SAID — claim 2

PyRIT records every protocol frame and projects the agent's **actions** into memory as tool-call pieces.

SO WHAT

scores are about behaviour, with provenance

BRING YOUR OWN MODEL — claim 3

The model behind the agent can be a **PyRIT target** behind a scoped relay; the harness and the model become independently testable.

SO WHAT

swap the model, hold the harness (or vice versa)

PEOPLE IN THE LOOP — claim 4

Operators watch live, approve or deny tool permissions, hold, cancel and close; decisions are attributed.

SO WHAT

a human can say no, mid-run

WHAT DOES NOT CHANGE

existing PyRIT surfaces

ATTACKS · CONVERTERS · SCORERS · MEMORY · CO-PYRIT CHAT

SOURCE

The one-sentence pitch in the design gist, split into its four load-bearing parts.

STEP 01 / 02

Four claims, each verifiable end to end

This is the whole pitch. Each claim maps to a chapter of this tour and to one or two pull requests in the stack; nothing here is aspirational.

PyRIT can red-team an agent, not just a model

AGENTS ARE TARGETS — claim 1

`AgentTarget` is one more **`PromptTarget`**. Attacks, converters and scorers run against it **unchanged**.

SO WHAT

your existing test suite runs against agents

DID, NOT SAID — claim 2

PyRIT records every protocol frame and projects the agent's **actions** into memory as tool-call pieces.

SO WHAT

scores are about behaviour, with provenance

BRING YOUR OWN MODEL — claim 3

The model behind the agent can be a **PyRIT target** behind a scoped relay; the harness and the model become independently testable.

SO WHAT

swap the model, hold the harness (or vice versa)

PEOPLE IN THE LOOP — claim 4

Operators watch live, approve or deny tool permissions, hold, cancel and close; decisions are attributed.

SO WHAT

a human can say no, mid-run

WHAT DOES NOT CHANGE — existing PyRIT surfaces

ATTACKS · CONVERTERS · SCORERS · MEMORY · CO-PYRIT CHAT

ONE NEW SEAM IN CORE

Targets gained an explicit **outcome** (PR 1) and a **single-inference** capability (PR 3); both are useful without agents.

STEP 02 / 02

Everything above the target is untouched

No new attack types, no new scorer types, no new memory schema. The agent arrives as a target; what it did arrives as pieces. That is why the stack can start with two small core PRs and why the rest is reviewable brick by brick.

`AgentTarget is a PromptTarget` — PR #17

```python
class AgentTarget(PromptTarget):
    """Send each new user turn to a fresh-per-conversation ACP execution."""

    requires_admin_creation: ClassVar[bool] = True
    # Over REST the server supplies this folder and every fixture directory must resolve
    upload_directory_parameter: ClassVar[str | None] = "allowed_fixture_directory"

    # Timeouts and evidence budgets are execution policy, not identity.
    _IDENTITY_EXCLUDED_FIELDS: ClassVar[set[str]] = {
        "startup_timeout_seconds",
```

pyrit/prompt_target/agent_target.py — 08-agent-framework · 383248e4a

— Claim — Unchanged PyRIT

A target is a recipe

WHERE THIS CHAPTER SITS



TOUR | DEEP DIVE

MODEL BINDING ‹ ›

harness default

The harness authenticates itself (e.g. COPILOT_GITHUB_TOKEN); PyRIT records that no binding exists.

default | openai | ollama

HARNESS PROFILE ‹ ›

copilot

`copilot --acp --stdio` · GitHub Copilot CLI.

copilot | AF host | generic

ENVIRONMENT TEMPLATE ‹ ›

docker

Container per execution, labelled and capability-dropped; the reference isolated provider.

docker | local | ACA

PERMISSION POLICY ‹ ›

deny

Default. Every tool permission request is refused; the agent is told so.

deny | allow | ask

TIMEOUTS · BUDGET ‹ ›

180 s · 300 s · 900 s

Turn deadline, idle expiry, lifetime, evidence and artifact budgets: execution policy, **excluded** from identity.

RECIPE · CONFIGURATION WITHOUT DEFAULTS, TIMEOUTS AND BUDGETS

```
environment_template: {"docker_network": "none", "environment": "docker", "image": "pyrit/agent-receipt"}
harness_profile: {"command": ["copilot", "--acp", "--stdio"], "harness_kind": "copilot"}
model_binding: {"target": null}
permission_policy: "deny"
```

IDENTITY = SHA256(RECIPE)

69c74bcc80039d1b…

provenance (kept, not hashed): harness=copilot-cli · environment=receipt-docker

Illustration of the rule in `AgentTarget._build_identifier` (configuration without defaults, timeouts and budgets); the bytes PyRIT hashes are pydantic's serialization, so the digits differ.

TRY IT

‹ › on any card; the hash below updates live.

STEP 01 / 06

Four parts you can swap

Which model answers, which agent runs and how it is launched, where it runs, and what happens when it asks for permission. Each part is reusable and registered by setup. Use the arrows to swap any of them.

PRESETS

Preset names are kept as provenance, never hashed.

The composition PR #21

```
112 class AgentTargetComposition(BaseModel):
    ··· 2 lines
115     model_config = ConfigDict(frozen=True, extra="forbid")
116
117     name: str
118     harness_profile: str
119     environment_template: str
120     model: str = ""
121     model_target: str | None = None
122     model_target_hash: str | None = Field(default=None, pattern=r"^[0-9a-f]{64}$")
123     configuration_name: str | None = Field(default=None, min_length=1, max_length=100)
124     expected_target_hash: str | None = Field(default=None, pattern=r"^[0-9a-f]{64}$")
    ··· 3 lines
```

pyrit/models/agent_setup.py 08-agent-framework · 383248e4a

— Part of the identity — Excluded: execution policy

A target is a recipe

MODEL BINDING
openai_chat

A registered `OpenAIChatTarget` answers every completion through the relay.

default · **openai** · ollama

HARNESS PROFILE
copilot

`copilot --acp --stdio` · GitHub Copilot CLI.

copilot · AF host · generic

ENVIRONMENT TEMPLATE
docker

Container per execution, labelled and capability-dropped; the reference isolated provider.

docker · local · ACA

PERMISSION POLICY
deny

Default. Every tool permission request is refused; the agent is told so.

deny · allow · ask

TIMEOUTS · BUDGET
180 s · 300 s · 900 s

Turn deadline, idle expiry, lifetime, evidence and artifact budgets: execution policy, **excluded** from identity.

RECIPE · CONFIGURATION WITHOUT DEFAULTS, TIMEOUTS AND BUDGETS

```
{
  "environment_template": {
    "docker_network": "none",
    "environment": "docker",
    "image": "pyrit/agent-receipt"
  },
  "harness_profile": {
    "command": [
      "copilot",
      "--acp",
      "--stdio"
    ],
    "harness_kind": "copilot"
```

IDENTITY = SHA256(RECIPE)

8ad2d2d758e8f9a2…

provenance (kept, not hashed): model=openai_chat · harness=copilot-cli · environment=receipt-docker

Illustration of the rule in `AgentTarget._build_identifier` (configuration without defaults, timeouts and budgets); the bytes PyRIT hashes are pydantic's serialization, so the digits differ.

WHY IT MATTERS

Regressions can be attributed to the model or to the scaffolding, not to "the agent".

STEP 02 / 06

Swap the model: a different system under test

Bind `openai_chat`. The harness is identical, but the recipe changed, so the identity changed: results from this target never mix with the previous one. This is **bring your own model**: the harness will reach the bound target only through PyRIT's relay.

How the relay works → Bring your own model, keep your own key

Identity excludes execution policy — PR #17

```
32  # Timeouts and evidence budgets are execution policy, not identity.
33  _IDENTITY_EXCLUDED_FIELDS: ClassVar[set[str]] = {
34      "startup_timeout_seconds",
35      "turn_timeout_seconds",
36      "idle_timeout_seconds",
37      "lifetime_seconds",
38      "cancellation_grace_seconds",
39      "approval_timeout_seconds",
40      "interactive_hold_seconds",
41      "max_evidence_bytes",
42      "max_artifact_bytes",
43  }
```

pyrit/prompt_target/agent_target.py — 08-agent-framework · 383248e4a

— Part of the identity — Excluded: execution policy

A target is a recipe

MODEL BINDING

openai_chat

A registered `OpenAIChatTarget` answers every completion through the relay.

default **openai** ollama

HARNESS PROFILE

copilot

`copilot --acp --stdio` · GitHub Copilot CLI.

copilot AF host generic

ENVIRONMENT TEMPLATE

local

A subprocess with a private home. **Not** a sandbox; requires an explicit acknowledgement.

docker **local** ACA

PERMISSION POLICY

deny

Default. Every tool permission request is refused: the agent is told so.

deny allow ask

TIMEOUTS · BUDGET

180 s · 300 s · 900 s

Turn deadline, idle expiry, lifetime, evidence and artifact budgets: execution policy, **excluded** from identity.

RECIPE · CONFIGURATION WITHOUT DEFAULTS, TIMEOUTS AND BUDGETS

```
{
  "environment_template": {
    "environment": "local",
    "local_execution_acknowledged": true
  },
  "harness_profile": {
    "command": [
      "copilot",
      "--acp",
      "--stdio"
    ],
    "harness_kind": "copilot"
  }
```

IDENTITY = SHA256(RECIPE)

77f031dc23229358…

provenance (kept, not hashed): model=openai_chat · harness=copilot-cli · environment=receipt-local

Illustration of the rule in `AgentTarget._build_identifier` (configuration without defaults, timeouts and budgets); the bytes PyRIT hashes are pydantic's serialization, so the digits differ.

PROVIDERS

`local` and `docker` ship with PyRIT; an ACA sandbox provider was built out of tree against the same contract.

STEP 03 / 06

Swap where it runs

Same model, same harness, now in a local process instead of a container. `environment` has no default: a target must say where it runs, and a non-isolated provider must be acknowledged.

The environment seam → Core owns policy, providers own resources

A template is never a live resource PR #17

```
225 class EnvironmentTemplate(BaseModel):
226     """
227     Reusable starting filesystem and access policy; never a running environment.
228
    … 5 lines
234     model_config = ConfigDict(frozen=True, extra="forbid")
235     environment: str
236     environment_settings: dict[str, JsonValue] = Field(default_factory=dict)
    … 4 lines
241     working_directory: str = "."
242     local_execution_acknowledged: bool = False
243     docker_network: str = "bridge"
    … 1 lines
```

pyrit/models/agent_execution.py 08-agent-framework · 383248e4a

— Part of the identity — Excluded: execution policy

A target is a recipe

MODEL BINDING

openai_chat

A registered `OpenAIChatTarget` answers every completion through the relay.

default **openai** ollama

HARNESS PROFILE

copilot

`copilot --acp --stdio` · GitHub Copilot CLI.

copilot AF host generic

ENVIRONMENT TEMPLATE

local

A subprocess with a private home. **Not** a sandbox; requires an explicit acknowledgement.

docker **local** ACA

PERMISSION POLICY

ask

An operator decides in Co-PyRIT; only valid for interactive sends.

deny allow **ask**

TIMEOUTS · BUDGET

180 s · 300 s · 900 s

Turn deadline, idle expiry, lifetime, evidence and artifact budgets: execution policy, **excluded** from identity.

RECIPE · CONFIGURATION WITHOUT DEFAULTS, TIMEOUTS AND BUDGETS

```
{
  "environment_template": {
    "environment": "local",
    "local_execution_acknowledged": true
  },
  "harness_profile": {
    "command": [
      "copilot",
      "--acp",
      "--stdio"
    ],
    "harness_kind": "copilot"
  }
```

IDENTITY = SHA256(RECIPE)

d1c9e9df0c4f0059…

provenance (kept, not hashed): model=openai_chat · harness=copilot-cli · environment=receipt-local

Illustration of the rule in `AgentTarget._build_identifier` (configuration without defaults, timeouts and budgets); the bytes PyRIT hashes are pydantic's serialization, so the digits differ.

NOT ENFORCEMENT

The policy is a deterministic protocol answer; isolation comes from the environment.

STEP 04 / 06

Permission policy is identity, and it defaults to deny

A target that **asks an operator** and one that **denies** every tool permission are different systems under test, so the policy is hashed. New targets refuse permission requests unless you say otherwise; `ask` is only valid for interactive chat and fails fast for attacks.

AgentPermissionPolicy — PR #17

```
70  class AgentPermissionPolicy(str, Enum):
71      """Deterministic permission responses, not a tool enforcement boundary."""
72
73      DENY = "deny"
74      ALLOW_ONCE = "allow_once"
75      ASK = "ask"
76
```

pyrit/models/agent_execution.py — 08-agent-framework · 383248e4a

— Part of the identity — Excluded: execution policy

A target is a recipe

MODEL BINDING

openai_chat

A registered `OpenAIChatTarget` answers every completion through the relay.

default · **openai** · ollama

HARNESS PROFILE

copilot

`copilot --acp --stdio` · GitHub Copilot CLI.

copilot · AF host · generic

ENVIRONMENT TEMPLATE

local

A subprocess with a private home. **Not** a sandbox; requires an explicit acknowledgement.

docker · **local** · ACA

PERMISSION POLICY

ask

An operator decides in Co-PyRIT; only valid for interactive sends.

deny · allow · **ask**

TIMEOUTS · BUDGET

60 s · 120 s · 600 s

Turn deadline, idle expiry, lifetime, evidence and artifact budgets: execution policy, **excluded** from identity.

RECIPE · CONFIGURATION WITHOUT DEFAULTS, TIMEOUTS AND BUDGETS

```
{
  "environment_template": {
    "environment": "local",
    "local_execution_acknowledged": true
  },
  "harness_profile": {
    "command": [
      "copilot",
      "--acp",
      "--stdio"
    ],
    "harness_kind": "copilot"
  }
```

IDENTITY = SHA256(RECIPE)

d1c9e9df0c4f0059…

provenance (kept, not hashed): model=openai_chat · harness=copilot-cli · environment=receipt-local

Illustration of the rule in `AgentTarget._build_identifier` (configuration without defaults, timeouts and budgets); the bytes PyRIT hashes are pydantic's serialization, so the digits differ.

RULE

`exclude_defaults=True, exclude=_IDENTITY_EXCLUDED_FIELDS` on the configuration dump.

STEP 05 / 06

Tighten a deadline: same identity

Timeouts, holds, evidence and artifact budgets are **execution policy**. Change them and the hash is unchanged: results stay comparable across a deadline tweak, while changing what runs or where creates a new target.

Building the identifier — PR #17

```
141 def _build_identifier(self) -> ComponentIdentifier:
142     # Defaulted fields are omitted so later optional settings never change existing ider
143     values = self.agent_configuration.model_dump(
144         mode="json", exclude_defaults=True, exclude=self._IDENTITY_EXCLUDED_FIELDS
145     )
146     return self._create_identifier(params={"configuration": values})
147
148 async def _get_normalized_conversation_async(
```

pyrit/prompt_target/agent_target.py — 08-agent-framework · 383248e4a

— Part of the identity — Excluded: execution policy

A target is a recipe

HOLD ONE, SWAP THE OTHER · FOUR DIFFERENT SYSTEMS UNDER TEST, EACH WITH ITS OWN IDENTITY AND ITS OWN RESULTS

HARNESS
copilot
MODEL
openai_chat
`d1c9e9df0c4f0059…`

HARNESS
copilot
MODEL
ollama_local
`ca7de726fc4ae5d2…`

HARNESS
agent_framework
MODEL
openai_chat
`13ce40bdabdc7d26…`

HARNESS
agent_framework
MODEL
ollama_local
`f77fb117b3793038…`

IN PRACTICE

Compose each cell from the same presets in YAML; names are provenance, so the matrix is auditable.

STEP 06 / 06

Hold one, swap the other

Two harnesses × two models = four systems under test, each with its own identity and its own results. Is the regression in the model or in the scaffolding? Run the row, run the column.

`Identity in depth → A target is a recipe, not a resource`

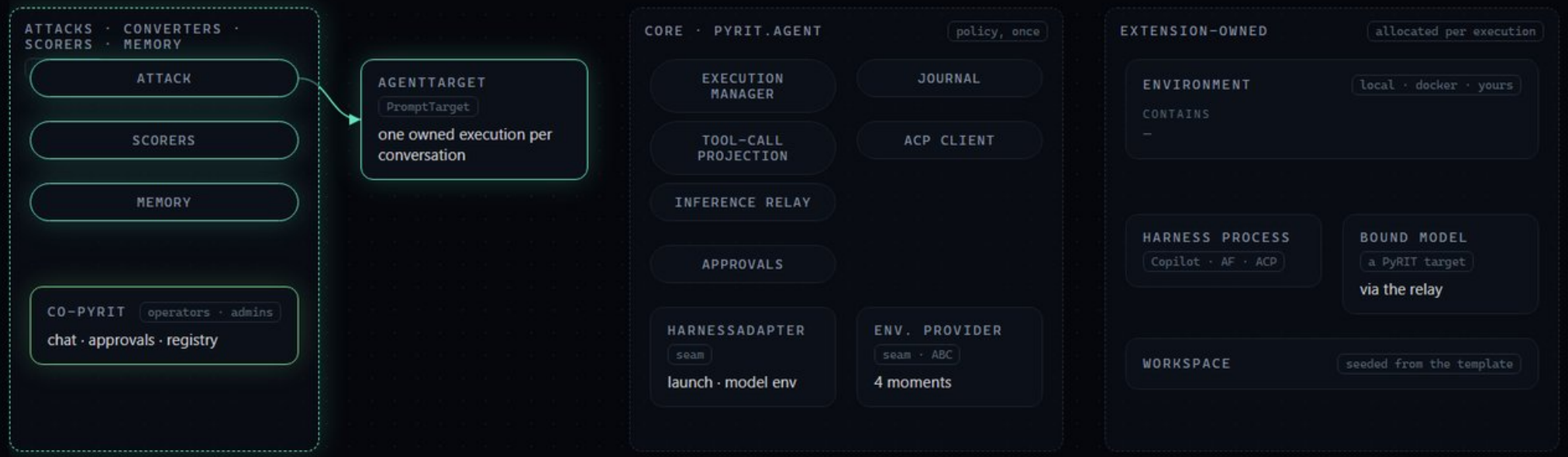
Provenance is kept outside identity PR #21

```
105 class AgentTargetProvenance(BaseModel):
106     """Origin retained outside the target's identity and execution recipe."""
107
108     origin: AgentTargetOrigin
109     presets: AgentPresetSelection = Field(default_factory=AgentPresetSelection)
110
```

pyrit/models/agent_setup.py 08-agent-framework · 383248e4a

— Part of the identity — Excluded: execution policy

Core owns policy, extensions own resources



READING THE MAP

Left to right: who asks → what owns the policy → who owns the resources.

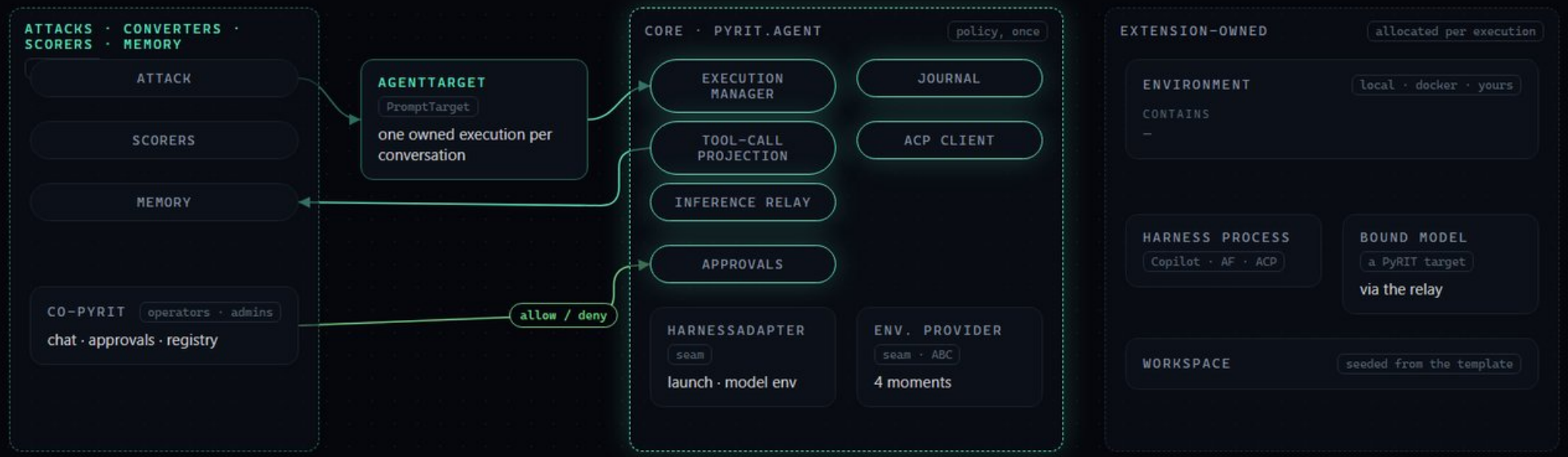
STEP 01 / 04

Nothing above the target changes

Attacks send messages; converters rewrite them; scorers read the pieces that come back; memory stores them. An agent target sits where any target sits. Operators and administrators meet the system in Co-PyRIT.

— PyRIT core — Seam · contract — Extension-owned resources — People

Core owns policy, extensions own resources



WHY ONCE

A provider cannot weaken redaction or budgets by accident, because it never sees them.

STEP 02 / 04

Core: the policy, implemented once

The execution manager owns timeouts, budgets, cancellation and reconciliation. The journal records every frame. The projection turns frames into pieces. Approvals route permission requests to policy or a person. The relay stands between the harness and any model. The ACP client speaks the protocol.

Core owns the lease budget — PR #20

```
589 lease = InferenceLease(
590     binding=binding,
591     record=record_async,
592     timeout_seconds=live.record.profile.turn_timeout_seconds,
593     max_calls=live.record.profile.max_inference_requests,
594     capture_content=live.record.profile.capture_inference_content,
595 )
596 self._relay.add(lease)
```

pyrit/agent/execution_manager.py — 08-agent-framework · 383248e4a

— PyRIT core — Seam · contract — Extension-owned resources — People

Core owns policy, extensions own resources



FOR REVIEWERS

A provider PR is reviewed against its conformance report, not against a reading of the ABC.

STEP 03 / 04

Two seams, both executable

`HarnessAdapter` : how a kind of harness is launched and pointed at a model.

`ExecutionEnvironmentProvider` : four moments (reserve, translate the relay URL, launch, close). Each has a conformance suite that drives a real implementation and reports one check per promise.

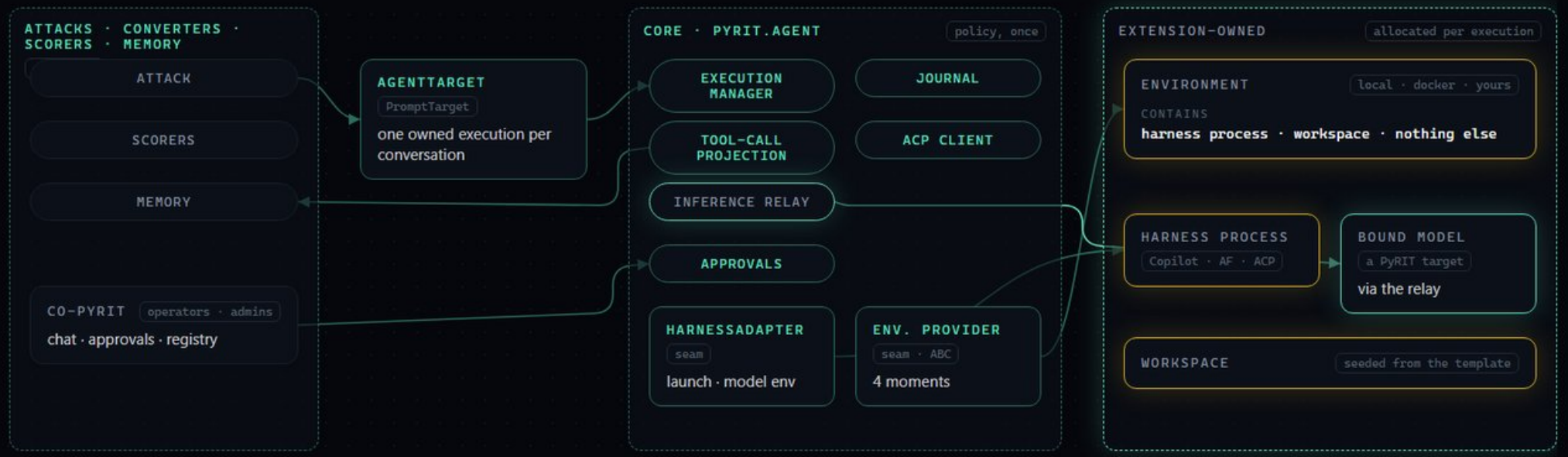
ExecutionSession · four moments — PR #17

```
99  def reserve(self) -> None:
100     """Record deterministic resource identifiers so a crash during allocation stays rec
    … 1 lines
102
103 async def inference_endpoint_async(self, *, relay_url: str) -> str:
104     """
    … 13 lines
118 @abstractmethod
119 async def launch_async(self, spec: ExecutionSpec) -> None:
120     """
    … 7 lines
128 @abstractmethod
129 async def close_async(self) -> None:
130     """Release exactly the owned resources, collect artifacts, and stay idempotent."""
```

pyrit/agent/environment.py — 08-agent-framework · 383248e4a

— PyRIT core — Seam · contract — Extension-owned resources — People

Core owns policy, extensions own resources



DISPOSED

Closing an execution releases exactly what the provider allocated, and refuses anything it did not create.

STEP 04 / 04

Extensions own what gets allocated

An environment (local process, Docker container, your sandbox) that contains the harness process and a workspace seeded from the template. The harness reaches a model only through the relay, with a lease, so no provider credential ever enters the environment.

Follow one turn → One turn, end to end

Ownership check before stop — PR #18

```python
async def _inspect_owned_container_async(self) -> dict[str, Any] | None:
    output = await self._docker_async(
        "container", "ls", "--all", "--quiet", "--filter", f"name=^/{self._container}$"
    )
    if not output.strip():
        return None
    info = json.loads(await self._docker_async("inspect", self._container))[0]
    labels = info["Config"].get("Labels", {})
    if (
        labels.get("pyrit.execution") != str(self.execution.id)
        or labels.get("pyrit.owner") != self.execution.owner_id
    ):
        raise RuntimeError("Refusing to stop a container whose ownership labels do not m
    return info
```

pyrit/agent/environments/docker.py — 08-agent-framework · 383248e4a

— PyRIT core — Seam · contract — Extension-owned resources — People

CHAPTER 04 · THE MAP

The same map, as the reviewers see it

The runtime-architecture diagram from the design gist. Everything on this tour is a zoom into one of its regions; the minimap in the header shows which.

SVG source and PNG on the `agents/stack-assets` branch; the gist embeds the same image.

Ask, watch, approve

Feed: connecting | ACP: not started | Environment: not provisioned

There are no messages in this conversation yet.

CO-PYRIT · CHAT · MOCK DRIVEN BY THE REAL JOURNAL

JOURNAL · EVENTS.JSONL · 0 / 28

FIXTURE

This replay is driven by `ask_events.jsonl`, a real journal captured with `permission_policy=ask`.

MOCK

The chat is a faithful mock; the real screenshots follow.

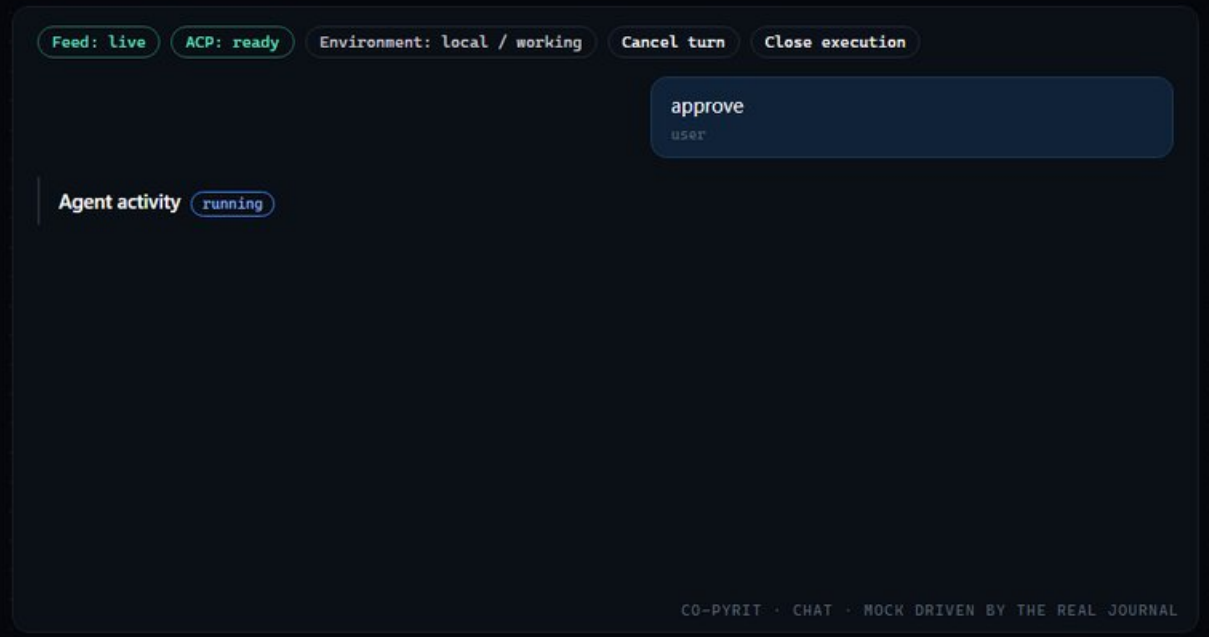
STEP 01 / 07

Pick an agent target like any other

In Co-PyRIT chat the target picker lists **`receipt_agent`** next to every other registered target. Model, harness profile and environment template are fixed for the conversation; a status strip shows the feed, the ACP connection and the environment.

— Outgoing · PyRIT — Incoming · the agent — Operator decision

Ask, watch, approve



JOURNAL · EVENTS.JSONL · 7 / 28

```
1  lifecycle   connection.state · connecting
2  outgoing    initialize
3  incoming    initialize ✓
4  outgoing    session/new
5  incoming    session/new ✓
6  lifecycle   connection.state · ready
7  outgoing    session/prompt · "approve"
```

STATUS STRIP

Feed: live · ACP: ready · Environment: local / working · Cancel turn · Close execution.

STEP 02 / 07

The first message provisions an execution

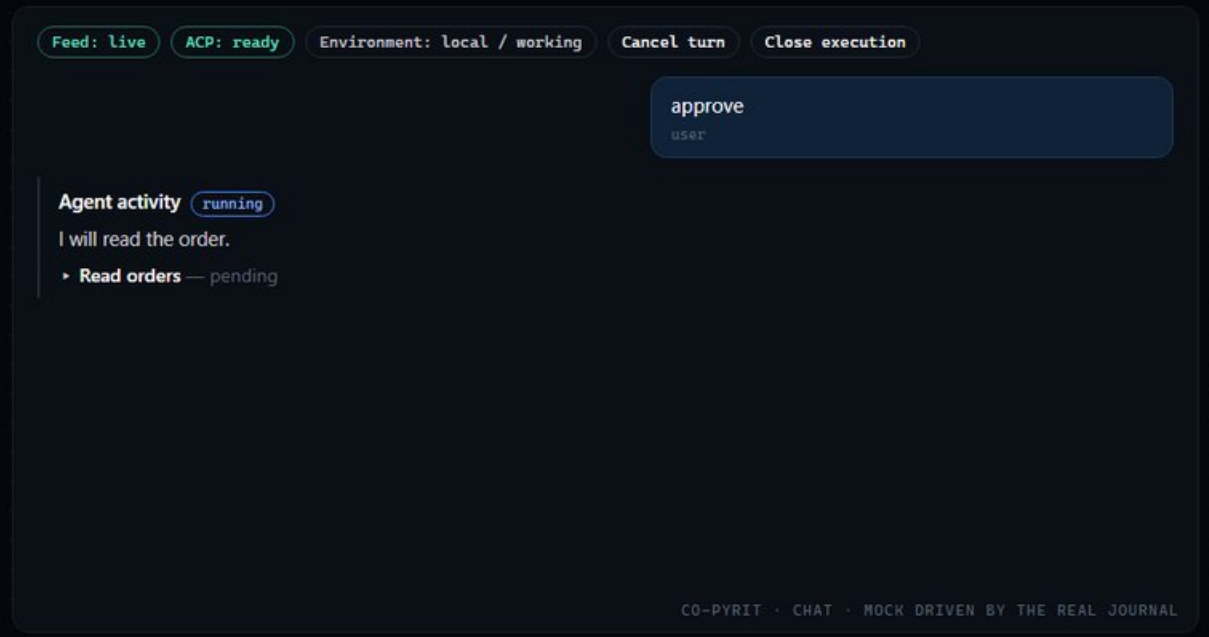
`initialize`, `session/new`, then `session/prompt`. The environment is allocated, the harness launched and connected; from now on the conversation reuses this execution. The journal on the right is the same data the chat streams over SSE.

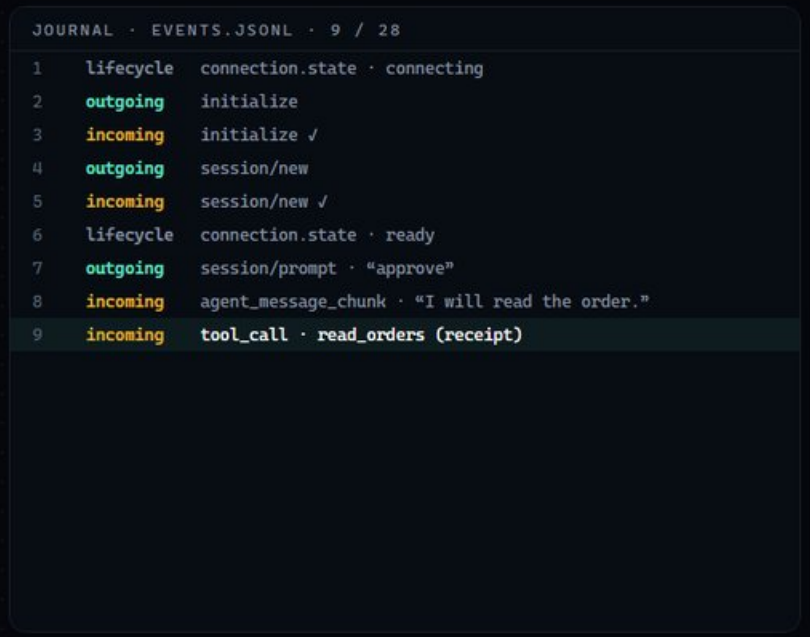
Conversation stream · state / reset / reconnect — PR #22

```
177 with manager.store.subscribe(conversation_id) as changed:
178     while monotonic() < deadline:
179         changed.clear()
180         if manager.is_closed:
181             yield _sse("reconnect", {"reason": "runtime_replaced"})
182             return
183         current = await get_conversation_execution_async(request, attack_result_id, conv
184         if current is not None and current.id ≠ identity:
185             cursor, identity = 0, current.id
186             yield _sse("reset", {"execution_id": str(identity)})
187         state = current.model_dump_json() if current else "null"
188         if state ≠ previous_state:
189             previous_state = state
190             yield f"event: state\ndata: {state}\n\n"
```

pyrit/backend/routes/executions.py — 08-agent-framework · 383248e4a

— Outgoing · PyRIT — Incoming · the agent — Operator decision





DID, NOT SAID

Only explicit, named tool calls with established results become memory pieces.

STEP 03 / 07

Watch it act

"I will read the order." Then a `tool_call` named `read_orders`. The activity panel renders each streamed update under the message as it arrives; nothing is invented from the agent's prose.

How the projection works → Recording what the agent did

— Outgoing · PyRIT — Incoming · the agent — Operator decision

Ask, watch, approve

Feed: live | ACP: ready | Environment: local / working | Awaiting 1 approval | Cancel turn | Close execution

approve
user

Agent activity running

I will read the order.

▸ **Read orders** — pending

Read orders.json
Harness requested approval. Expires in 120 s.

Allow once | Deny

CO-PYRIT · CHAT · MOCK DRIVEN BY THE REAL JOURNAL

JOURNAL · EVENTS.JSONL · 11 / 28

1	lifecycle	connection.state · connecting
2	outgoing	initialize
3	incoming	initialize ✓
4	outgoing	session/new
5	incoming	session/new ✓
6	lifecycle	connection.state · ready
7	outgoing	session/prompt · "approve"
8	incoming	agent_message_chunk · "I will read the order."
9	incoming	tool_call · read_orders (receipt)
10	incoming	session/request_permission · Read orders.json
11	lifecycle	**permission.pending**

TRY IT

Click **Allow once** in the mock.

ATTACKS

Automated attacks cannot use `ask`; they fail fast instead of hanging.

STEP 04 / 07

The agent asks; the execution is held

`session/request_permission` for "Read orders.json". With the `ask` policy the runtime turns it into an approval and holds the execution: **Allow once** or **Deny** appear under the tool call, with the deadline. The harness had to offer a `reject_once` option, so a denial can be expressed as a denial.

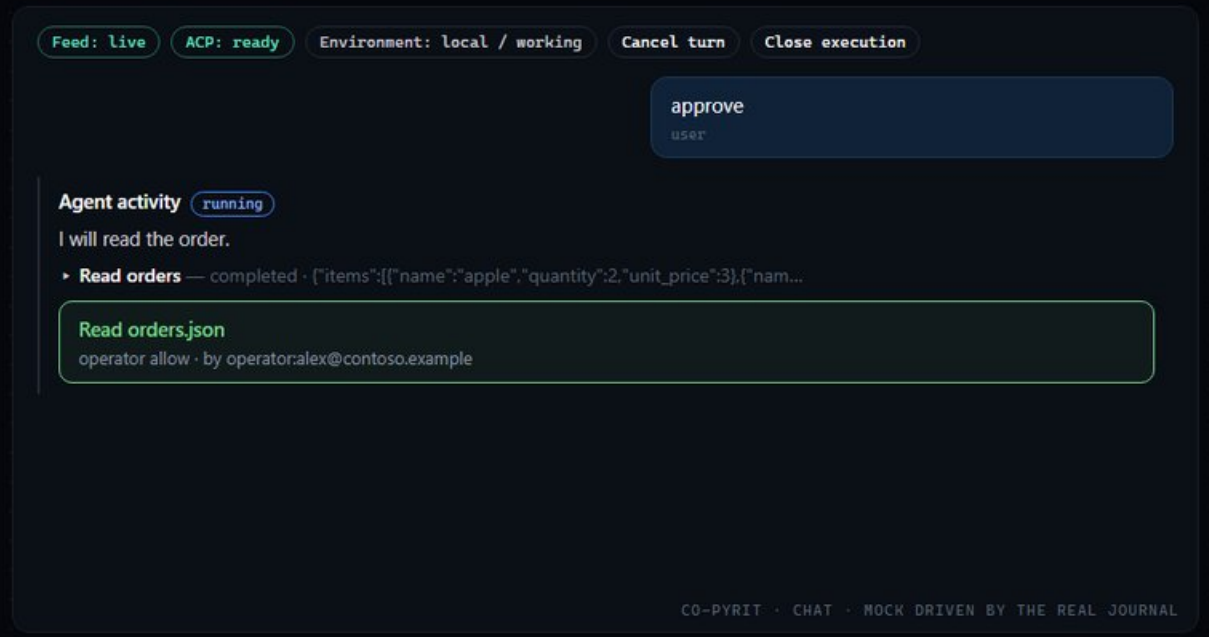
ACP client · request_permission — PR #24

```
105 async def request_permission(  # pyrit-async-suffix-exempt
    … 3 lines
109         return RequestPermissionResponse(outcome=DeniedOutcome(outcome="cancelled"))
110     if self.policy == AgentPermissionPolicy.ASK:
111         if self.approvals is None:
112             raise RequestError.invalid_params({"details": "Interactive approval owner is
113         if not any(option.kind == "reject_once" for option in options):
114             raise RequestError.invalid_params(
    … 1 lines
116             )
117         option_id = await self.approvals.request_async(
118             tool_call_id=tool_call.tool_call_id,
    … 6 lines
```

pyrit/agent/acp_connection.py — 08-agent-framework · 383248e4a

— Outgoing · PyRIT — Incoming · the agent — Operator decision

Ask, watch, approve



JOURNAL · EVENTS.JSONL · 14 / 28

2	outgoing	initialize
3	incoming	initialize ✓
4	outgoing	session/new
5	incoming	session/new ✓
6	lifecycle	connection.state · ready
7	outgoing	session/prompt · “approve”
8	incoming	agent_message_chunk · “I will read the order.”
9	incoming	tool_call · read_orders (receipt)
10	incoming	session/request_permission · Read orders.json
11	lifecycle	permission.pending
12	lifecycle	permission.resolved · operator_allow · operator:alex@conto…
13	outgoing	permission reply · yes (selected)
14	incoming	**tool_call_update · completed · {"items":[{"name":"apple","…**

ATTRIBUTION

Visible to everyone with access to the attack by default; `show_agent_operator_identity: false` shows roles instead.

STEP 05 / 07

An operator allows it, once

`permission.resolved · operator_allow` names the operator. The reply goes back to the agent as the `allow_once` option it offered, the tool runs, and `tool_call_update` carries the real result. Decisions are server-owned, so a refresh or a second tab cannot lose or double-answer one.

`Approvals in depth → Watch it, approve it, hold it`

AgentApprovals · decide — PR #24

```
136     raise ValueError("Permission request is no longer pending")
137 if decision not in ("operator_allow", "operator_deny", "timeout_deny", "cancelled"):
138     raise ValueError("Unsupported permission decision")
139 if decision.startswith("operator") and datetime.now(UTC) ≥ approval.expires_at:
140     raise ValueError("Permission request has expired")
141 kind = "allow_once" if decision == "operator_allow" else "reject_once"
142 option = next((item["option_id"] for item in approval.options if item["kind"] == kind),
143 if option is None and decision == "operator_allow":
144     raise ValueError("Harness did not offer a one-time approval")
145 approval.decision, approval.option_id, approval.actor = decision, option, actor
146 try:
147     await self.changed(approval)
148 except BaseException:
149     approval.decision = approval.option_id = approval.actor = None
```

pyrit/agent/approvals.py — 08-agent-framework · 383248e4a

— Outgoing · PyRIT — Incoming · the agent — Operator decision

Ask, watch, approve

Feed: live | ACP: ready | Environment: local / idle | Close execution

approve
user

Agent activity completed

I will read the order.Now I will write the receipt.Receipt total: 31

- **Read orders** — completed · {"items":[{"name":"apple","quantity":2,"unit_price":3},{"nam…
- **Write receipt** — completed · {"total":31}

Read orders.json
operator allow · by operator:alex@contoso.example

I will read the order.Now I will write the receipt.Receipt total: 31
assistant · tool calls stored as pieces

scoring · **MessageToolCallScorer** read read_orders → {"total": 31} · outcome **completed**

CO-PYRIT · CHAT · MOCK DRIVEN BY THE REAL JOURNAL

JOURNAL · EVENTS.JSONL · 20 / 28

8	incoming	agent_message_chunk · "I will read the order."
9	incoming	tool_call · read_orders (receipt)
10	incoming	session/request_permission · Read orders.json
11	lifecycle	permission.pending
12	lifecycle	permission.resolved · operator_allow · operator:alex@conto…
13	outgoing	permission reply · yes (selected)
14	incoming	tool_call_update · completed · {"items":[{"name":"apple","…
15	incoming	agent_message_chunk · "Now I will write the receipt."
16	incoming	tool_call · write_receipt (write)
17	incoming	tool_call_update · completed · {"total":31}
18	incoming	agent_message_chunk · "Receipt total: 31"
19	incoming	result · stopReason end_turn
20	lifecycle	**turn_finished · completed**

OUTCOME

A turn with tool effects and no text would still be `completed`, which is why PR 1 made outcomes explicit.

STEP 06 / 07

"Receipt total: 31", and a score about what it did

The turn completes. The transcript stores the assistant text plus the projected tool pieces; Co-PyRIT auto-scores completed assistant and tool pieces with whatever scorer you configured. The agent's words and its actions are both evidence now.

What gets auto-scored — PR #19

```
80  def _is_scorable_response(piece: MessagePiece) -> bool:
81      return (
82          piece.role in ("assistant", "tool")
83          and not piece.prompt_metadata.get(RequestTraceContext.REQUEST_METADATA_KEY)
84          and TargetOutcome.from_metadata(piece.prompt_metadata) is TargetOutcome.COMPLETE
85      )
86
```

pyrit/backend/services/message_send_service.py 08-agent-framework · 383248e4a

— Outgoing · PyRIT — Incoming · the agent — Operator decision

Ask, watch, approve

Feed: live | ACP: ready | Environment: local / idle | Close execution

approve
user

Agent activity completed

I will read the order.Now I will write the receipt.Receipt total: 31

- **Read orders** — completed · {"items":[{"name":"apple","quantity":2,"unit_price":3},{"nam…
- **Write receipt** — completed · {"total":31}

Read orders.json
operator allow · by operator:alex@contoso.example

I will read the order.Now I will write the receipt.Receipt total: 31
assistant · tool calls stored as pieces

Add one orange to the order
user

CO-PYRIT · CHAT · MOCK DRIVEN BY THE REAL JOURNAL

JOURNAL · EVENTS.JSONL · 26 / 28

14	incoming	tool_call_update · completed · {"items":[{"name":"apple","…
15	incoming	agent_message_chunk · "Now I will write the receipt."
16	incoming	tool_call · write_receipt (write)
17	incoming	tool_call_update · completed · {"total":31}
18	incoming	agent_message_chunk · "Receipt total: 31"
19	incoming	result · stopReason end_turn
20	lifecycle	turn_finished · completed
21	outgoing	session/prompt · "Add one orange to the order"
22	incoming	tool_call · read_orders (receipt)
23	incoming	tool_call_update · completed · {"total":36}
24	incoming	agent_message_chunk · "Receipt total: 36"
25	incoming	result · stopReason end_turn
26	lifecycle	**turn_finished · completed**

LIFETIME

The execution ends on explicit close, idle expiry, its maximum lifetime, a turn deadline, or runtime shutdown.

STEP 07 / 07

The second message reuses the execution

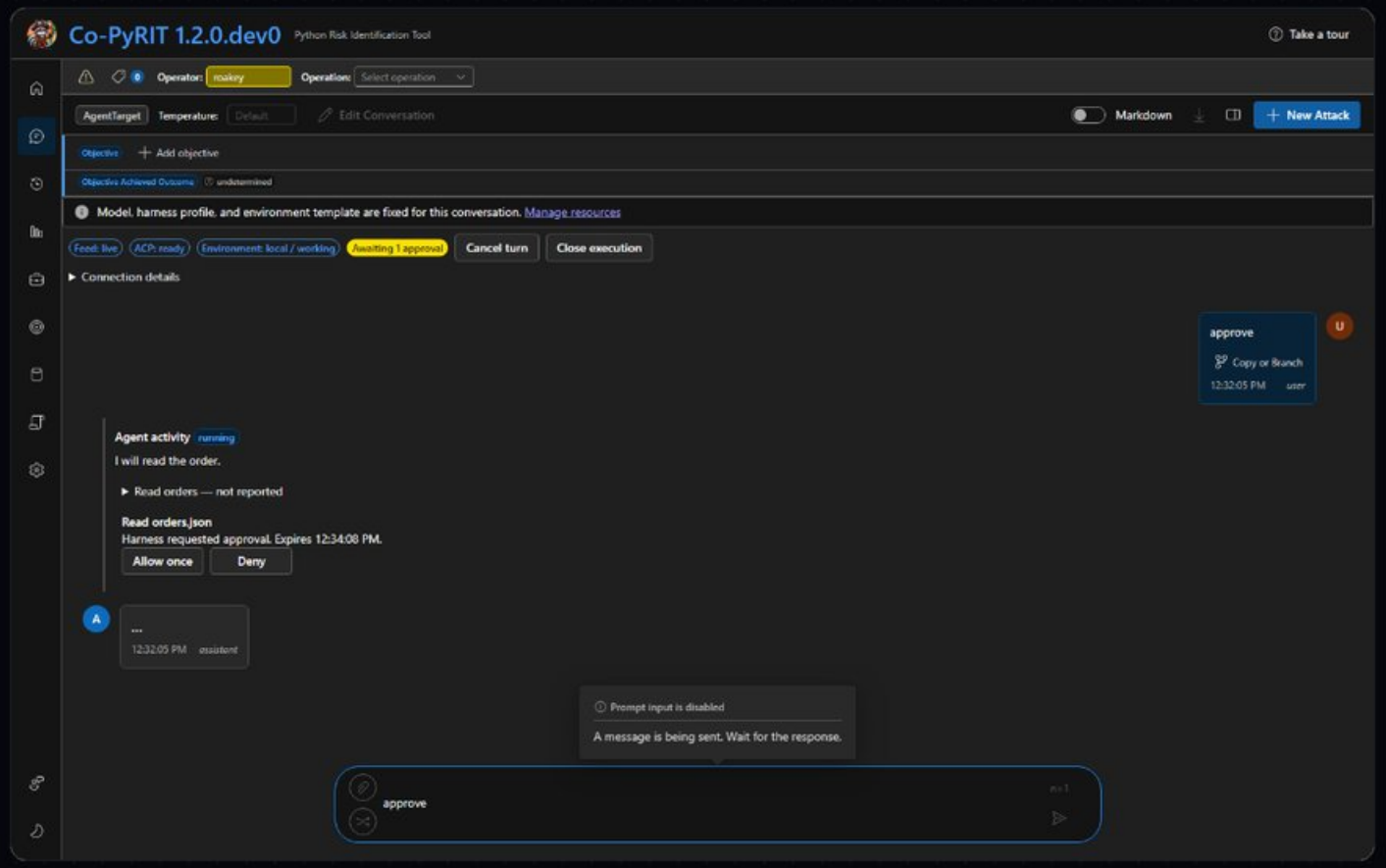
"Add one orange to the order" changes a file the agent itself wrote earlier: the total goes from 31 to 36. Same execution, same workspace, same journal; a stateful system under test, not a stateless endpoint.

— Outgoing · PyRIT — Incoming · the agent — Operator decision

CHAPTER 05 · REAL CAPTURE

It is real: the permission moment

The same flow in the actual Co-PyRIT UI, captured headlessly against this branch with the scripted agent and the local provider. Status strip, activity, the pending approval with **Allow once / Deny**, and the deadline.

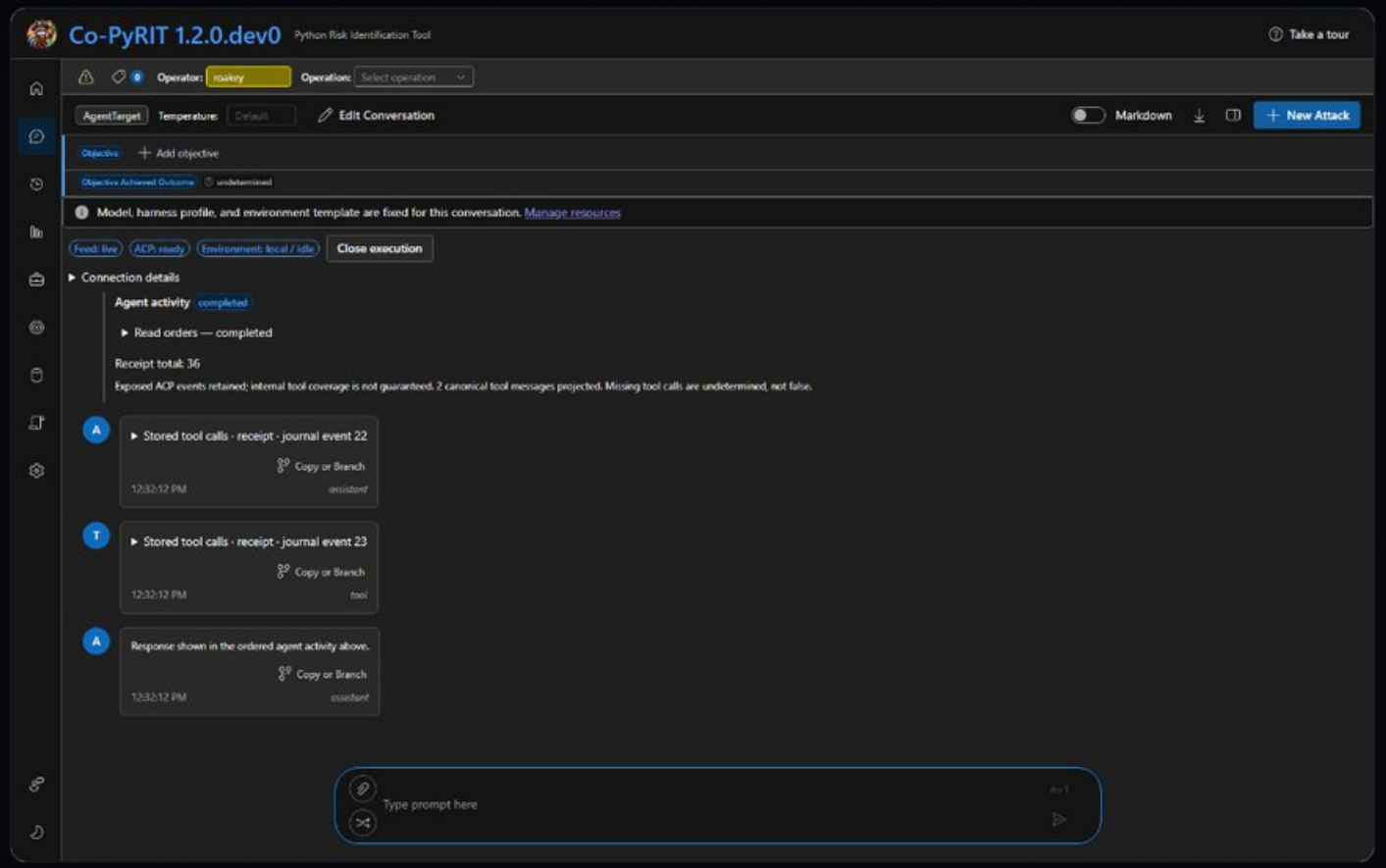


`fixtures/real/capture.mjs` reproduces this capture; no credentials involved.

CHAPTER 05 · REAL CAPTURE

It is real: two turns later

Second turn complete: **`Receipt total: 36`**, the stored tool-call pieces (“journal event 22 · 23”), and the honest footnote: *exposed ACP events retained; missing tool calls are undetermined, not false*.



The footnote is the projection's contract in one sentence.

Create, gate, govern, clean up

NEW TARGET → AGENT (ACP) — administrators only

MODEL	HARNESS	ENVIRONMENT
registered target · wire API inferred	preset or custom · command · policy	preset or custom · provider · image

SAFE DEFAULTS — new targets

PERMISSION POLICY
—

LOCAL PROVIDER
—

ASK
—

SETTINGS — configuration

OPERATOR IDENTITY
—

INFERENCE BUDGET
—

CONFIGURATION REPLACEMENT
—

CREATION GATE — server-enforced

WHO
—

INPUT
—

FIXTURES
—

REGISTRY → AGENT EXECUTIONS — resource view

STATES
—

ACTIONS
—

WHAT THE CATALOG SHOWS — to every authenticated user

SHOWS
—

NEVER
—

REAL SCREENSHOT

Next step shows the actual form.

STEP 01 / 06

One form, three sections, in that order

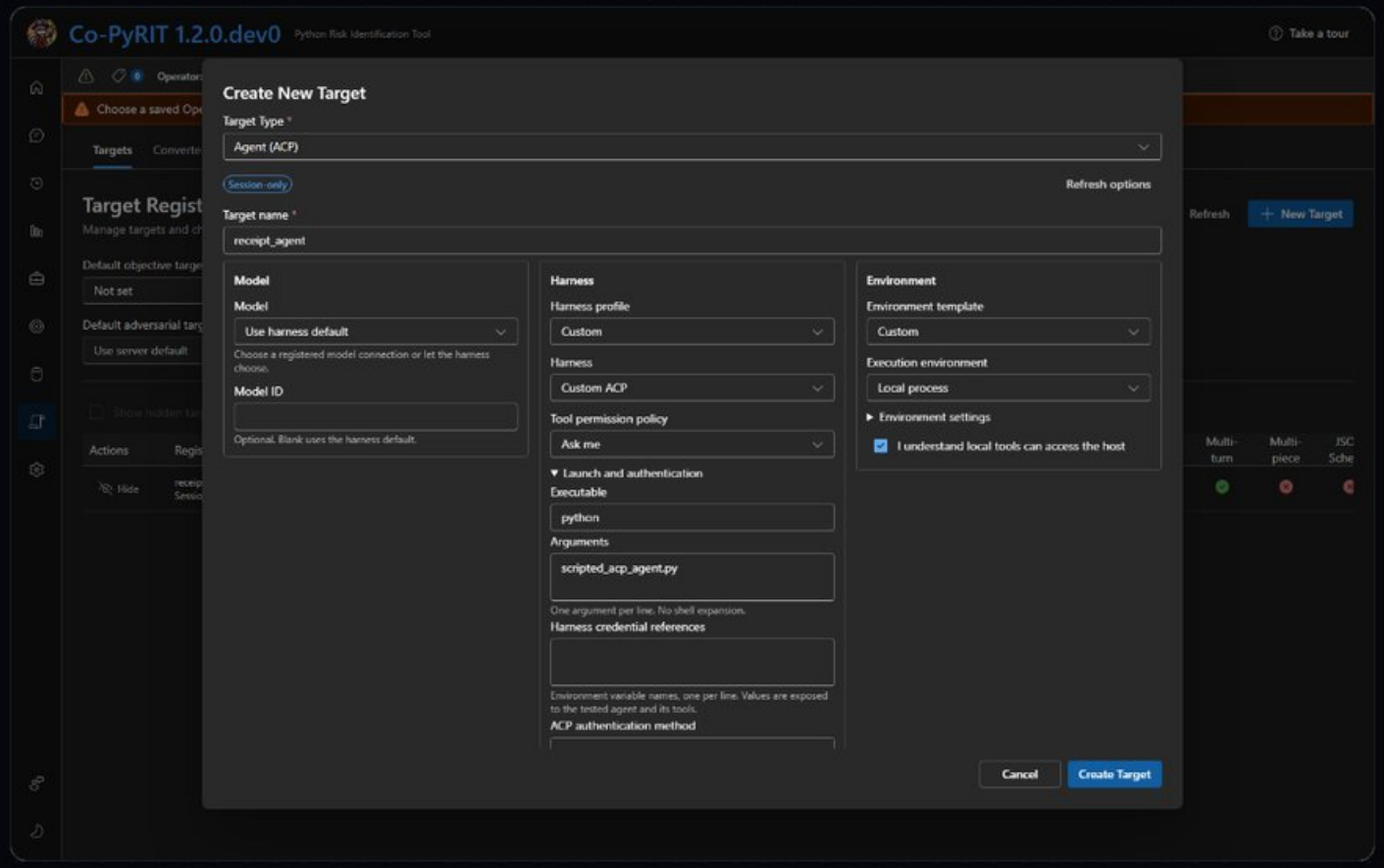
Model picks a registered connection for BYOM or lets the harness authenticate itself. *Harness* and *Environment* offer registered presets; choosing one locks its fields and records the names as provenance, *Customize* copies the values and drops the provenance. Nothing in the form ever resolves a credential: credential fields hold variable **names**.

— Administrator surface — Refused — Visible to everyone

CHAPTER 06 · REAL CAPTURE

The real form

Captured from this branch: Model · Harness · Environment, *Ask me* selected for the demo, *Local process* with the acknowledgement that local tools can access the host.



Preset dropdowns are empty here because the demo backend registers no presets; with initializers they list named profiles and templates.

Create, gate, govern, clean up

NEW TARGET → AGENT (ACP) — administrators only

MODEL	HARNESS	ENVIRONMENT
registered target · wire API inferred	preset or custom · command · policy	preset or custom · provider · image

SAFE DEFAULTS — new targets

PERMISSION POLICY
—

LOCAL PROVIDER
—

ASK
—

SETTINGS — configuration

OPERATOR IDENTITY
—

INFERENCE BUDGET
—

CONFIGURATION REPLACEMENT
—

CREATION GATE — server-enforced

WHO
administrators · requires_admin_creation

INPUT
one pydantic model · JsonParameterValue

FIXTURES
inside target_upload_directory

REGISTRY → AGENT EXECUTIONS — resource view

STATES
—

ACTIONS
—

WHAT THE CATALOG SHOWS — to every authenticated user

SHOWS
—

NEVER
—

WHY

Every execution launches a process or container with credentials the server holds, regardless of parameters.

STEP 02 / 06

Admin-only, one validated object, confined fixtures

`requires_admin_creation` is a class marker checked by the create-target service next to upstream's `code_execution_parameter` and `server_credential_parameter`. `agent_configuration` crosses REST as one JSON object because the model opts in with `JsonParameterValue`; exactly one parameter in the repository does. A fixture directory must resolve inside the server's `target_upload_directory`.

AgentTarget · class markers — PR #17

```
26  """Send each new user turn to a fresh-per-conversation ACP execution."""
27
28  requires_admin_creation: ClassVar[bool] = True
29  # Over REST the server supplies this folder and every fixture directory must resolve ins
30  upload_directory_parameter: ClassVar[str | None] = "allowed_fixture_directory"
31
```

pyrit/prompt_target/agent_target.py — 08-agent-framework · 383248e4a

— Administrator surface — Refused — Visible to everyone

Create, gate, govern, clean up

NEW TARGET → AGENT (ACP)

administrators only

MODEL	HARNESS	ENVIRONMENT
registered target · wire API inferred	preset or custom · command · policy	preset or custom · provider · image

SAFE DEFAULTS

new targets

PERMISSION POLICY
deny

LOCAL PROVIDER
explicit acknowledgement required

ASK
interactive sends only

SETTINGS

configuration

OPERATOR IDENTITY
—

INFERENCE BUDGET
—

CONFIGURATION REPLACEMENT
—

CREATION GATE

server-enforced

WHO
administrators · requires_admin_creation

INPUT
one pydantic model · JsonParameterValue

FIXTURES
inside target_upload_directory

REGISTRY → AGENT EXECUTIONS

resource view

STATES
—

ACTIONS
—

WHAT THE CATALOG SHOWS

to every authenticated user

SHOWS
—

NEVER
—

IDENTITY

Policy is part of the target's identity: deny-targets and ask-targets never share results.

STEP 03 / 06

Deny by default; local must be acknowledged

New targets, in Python and in the UI, refuse tool permission requests. `allow_once` and `ask` are explicit choices; `ask` is only valid for interactive chat sends and fails fast for attacks. Choosing a provider that is not isolated requires ticking *I understand local tools can access the host*.

HarnessProfile defaults — PR #17

```
209 class HarnessProfile(BaseModel):
210     """Reusable harness behavior and authentication references, independent of placement
211
212     model_config = ConfigDict(frozen=True, extra="forbid")
213     command: tuple[str, ...] = ("copilot", "--acp", "--stdio")
214     harness_kind: str = AgentHarnessKind.COPILOT
215     credential_env: tuple[str, ...] = ()
216     authentication_method: str | None = None
217     permission_policy: AgentPermissionPolicy = AgentPermissionPolicy.DENY
218     inference_requirements: InferenceRequirements = Field(default_factory=InferenceRequi
```

pyrit/models/agent_execution.py — 08-agent-framework · 383248e4a

— Administrator surface — Refused — Visible to everyone

Create, gate, govern, clean up

NEW TARGET → AGENT (ACP)

administrators only

MODEL	HARNESS	ENVIRONMENT
registered target · wire API inferred	preset or custom · command · policy	preset or custom · provider · image

SAFE DEFAULTS

new targets

PERMISSION POLICY
deny

LOCAL PROVIDER
explicit acknowledgement required

ASK
interactive sends only

SETTINGS

configuration

OPERATOR IDENTITY
—

INFERENCE BUDGET
—

CONFIGURATION REPLACEMENT
—

CREATION GATE

server-enforced

WHO
administrators · requires_admin_creation

INPUT
one pydantic model · JsonParameterValue

FIXTURES
inside target_upload_directory

REGISTRY → AGENT EXECUTIONS

resource view

STATES
starting · idle · working · held · closing · closed · cleanup_failed

ACTIONS
cancel turn · close · retry cleanup

WHAT THE CATALOG SHOWS

to every authenticated user

SHOWS
—

NEVER
—

OBSERVATION

Watching an execution never renews its lifetime; only work does.

STEP 04 / 06

Executions are resources: listed, closable, reconciled

`Registry → Agent executions` lists what the backend owns with its state (`starting · idle · working · held · closing · closed · cleanup_failed`), its profile and environment, and the last turn. Administrators can cancel a turn or close an execution, including retrying failed cleanup. Interrupted executions are reconciled on restart through the same provider path.

AgentExecutionState PR #17

```
312 class AgentExecutionState(str, Enum):
313     """Resource status, separate from turn outcome and attack score."""
314
315     STARTING = "starting"
316     IDLE = "idle"
317     WORKING = "working"
318     HELD = "held"
319     CLOSING = "closing"
320     CLOSED = "closed"
321     CLEANUP_FAILED = "cleanup_failed"
```

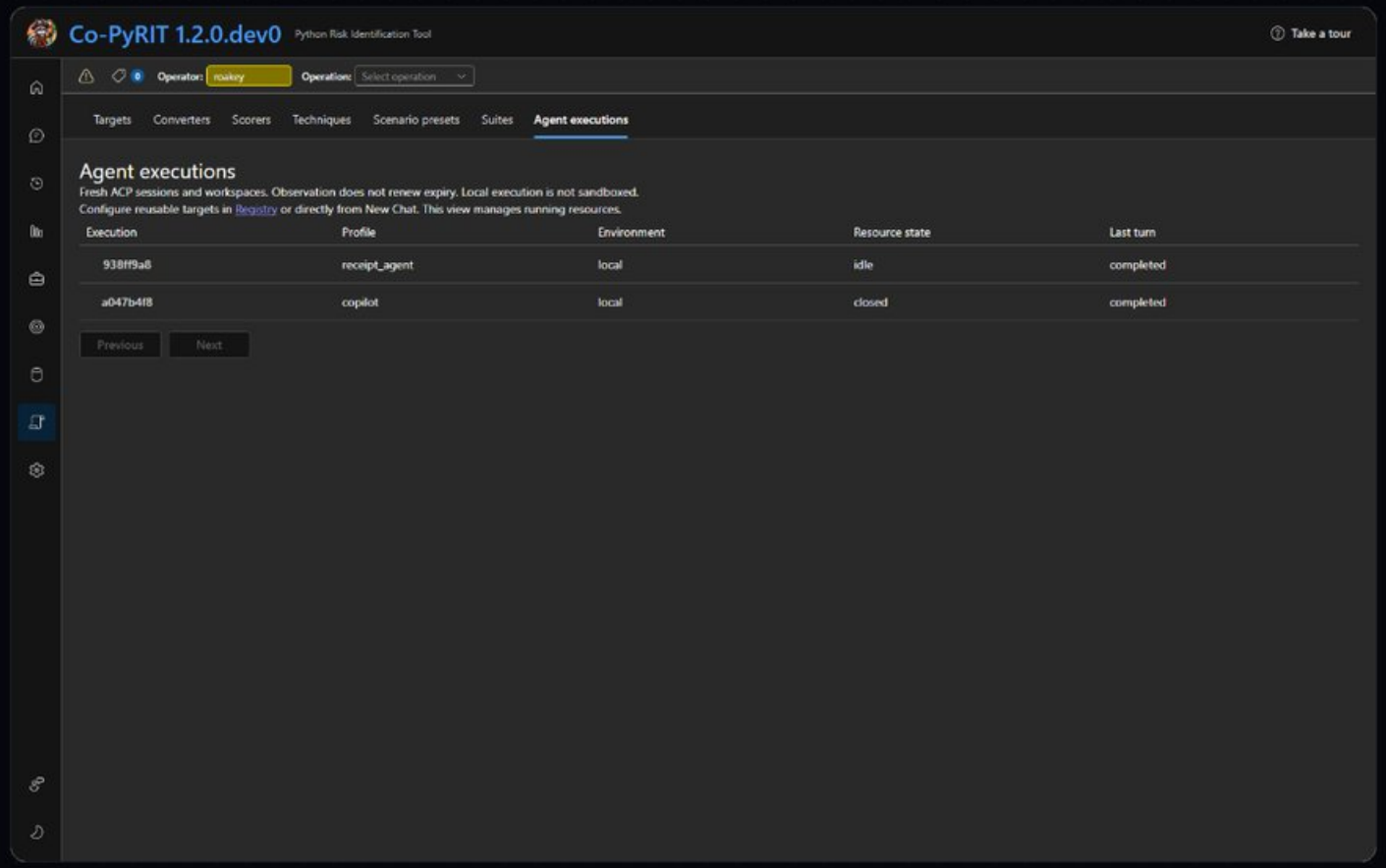
pyrit/models/agent_execution.py 08-agent-framework · 383248e4a

— Administrator surface — Refused — Visible to everyone

CHAPTER 06 · REAL CAPTURE

The real registry

Captured after the chat above: the `receipt_agent` execution idle on `local` with its last turn completed, next to an older, closed one. This is the administrator's resource view, separate from the chat.



Observation does not renew expiry; the idle execution closes on its idle timeout or when an administrator closes it.

Create, gate, govern, clean up

NEW TARGET → AGENT (ACP)

administrators only

MODEL	HARNESS	ENVIRONMENT
registered target · wire API inferred	preset or custom · command · policy	preset or custom · provider · image

SAFE DEFAULTS

new targets

PERMISSION POLICY
deny

LOCAL PROVIDER
explicit acknowledgement required

ASK
interactive sends only

SETTINGS

configuration

OPERATOR IDENTITY
show (default) · or roles

INFERENCE BUDGET
100 default · 1,000 cap

CONFIGURATION REPLACEMENT
blocked while any execution is live

CREATION GATE

server-enforced

WHO
administrators · requires_admin_creation

INPUT
one pydantic model · JsonParameterValue

FIXTURES
inside target_upload_directory

REGISTRY → AGENT EXECUTIONS

resource view

STATES
starting · idle · working · held · closing · closed · cleanup_failed

ACTIONS
cancel turn · close · retry cleanup

WHAT THE CATALOG SHOWS

to every authenticated user

SHOWS
—

NEVER
—

PRESENTATION ONLY

Masking identity changes nothing stored; the record and the admin view keep it.

STEP 05 / 06

Three knobs worth knowing

`show_agent_operator_identity` (default **show**) controls whether decisions name the operator or just the role. Per-execution inference budgets default to 100 requests and cap at 1,000. Live executions, including idle and held chats, block configuration replacement exactly as active sends and scenario runs do.

Live executions block replacement PR #22

```
246 def _has_active_work(self) -> bool:
247     """Return whether any admitted or background runtime operation remains."""
248     service = peek_scenario_run_service()
249     agents = peek_agent_execution_manager()
250     return bool(
251         (service and service.has_active_work())
252         or (agents and agents.has_live_executions)
253         or self.operations
254         or outstanding_estimates()
255         or has_active_manual_sends()
256         or has_active_analytics_work()
257     )
```

pyrit/backend/services/runtime_lifecycle.py 08-agent-framework · 383248e4a

— Administrator surface — Refused — Visible to everyone

Create, gate, govern, clean up



NEW TARGET → AGENT (ACP)

administrators only

MODEL	HARNESS	ENVIRONMENT
registered target · wire API inferred	preset or custom · command · policy	preset or custom · provider · image

SAFE DEFAULTS

new targets

PERMISSION POLICY
deny

LOCAL PROVIDER
explicit acknowledgement required

ASK
interactive sends only

SETTINGS

configuration

OPERATOR IDENTITY
show (default) · or roles

INFERENCE BUDGET
100 default · 1,000 cap

CONFIGURATION REPLACEMENT
blocked while any execution is live

CREATION GATE

server-enforced

WHO
administrators · requires_admin_creation

INPUT
one pydantic model · JsonParameterValue

FIXTURES
inside target_upload_directory

REGISTRY → AGENT EXECUTIONS

resource view

STATES
starting · idle · working · held · closing · closed · cleanup_failed

ACTIONS
cancel turn · close · retry cleanup

WHAT THE CATALOG SHOWS

to every authenticated user

SHOWS
credential variable names · fixture path · presets

NEVER
credential values · image digest · process identities (conversation routes)

DECLINED REVIEW ITEM

Hiding the names was considered and declined: operators need to know which variables a profile expects.

STEP 06 / 06

What everyone can see, and what no one can

The target catalog shows credential variable **names** and the resolved fixture path to every authenticated user: that is the authoring contract. Values never appear anywhere: not in the catalog, not in the journal (redacted before writing), not in conversation routes (which omit owner, provider and process identities).

Masking operator identity — PR #24

```python
def _operator_role(actor: str | None) -> str | None:
    if actor is None or actor in _RUNTIME_ACTORS:
        return actor
    return "operator"


def _mask_event_actors(event: AgentExecutionEvent) -> AgentExecutionEvent:
    payload = dict(event.payload)
    actor = payload.get("actor")
    if isinstance(actor, str):
        payload["actor"] = _operator_role(actor)
    approval = payload.get("approval")
    if isinstance(approval, dict):
... 5 lines
```

pyrit/backend/routes/executions.py — 08-agent-framework · 383248e4a

— Administrator surface — Refused — Visible to everyone

Make PyRIT use my sandbox

EXECUTIONENVIRONMENTPROVIDER · THE FOUR MOMENTS

01 reserve()
Record deterministic resource identifiers before anything exists, so a crash mid-allocation stays reconcilable.

02 inference_endpoint_async(relay_url)
Translate the relay URL into your network namespace. Must not acquire resources.

03 launch_async(spec)
Seed the workspace, acquire resources, start the harness. Only variable names on command lines; values through your own channel.

04 close_async()
Release exactly what you own, collect artifacts, stay idempotent, refuse anything you did not create.

CONFORMANCEREPORT · PROVIDER 'LOCAL' · REAL RUN — NOT RUN

- ✓ template names this provider
- ✓ provider is registered under its name
- ✓ unsandboxed providers require consent and isolated providers do not
- ✓ create_session returns an ExecutionSession
- ✓ acp_reader is unavailable before launch
- ✓ acp_writer is unavailable before launch
- ✓ stderr is unavailable before launch
- ✓ inference_endpoint_async translates at least one relay address and rejects others with ValueError
- ✓ launch_async starts the harness
- ✓ ACP byte streams are full-duplex and order-preserving
- ✓ closing ACP input ends the output stream
- ✓ stderr is readable to end of stream
- ✓ credential values never reach the record
- ✓ provenance is a string mapping
- ✓ close_async releases resources
- ✓ close_async is idempotent
- ✓ a repeated close_async leaves the record unchanged

POLICY STAYS IN CORE

Timeouts, budgets, redaction and the protocol are not yours to implement, or to weaken.

STEP 01 / 04

Four moments, two byte streams

A provider subclasses `ExecutionEnvironmentProvider` (an `info` description, template validation, `create_session`) and returns an `ExecutionSession` with four lifecycle methods and three streams: ACP reader, ACP writer, stderr. Nothing here speaks ACP; you move bytes and honour the moments.

ExecutionSession · four moments — PR #17

```
99  def reserve(self) -> None:
100     """Record deterministic resource identifiers so a crash during allocation stays reco
    ... 1 lines
102
103 async def inference_endpoint_async(self, *, relay_url: str) -> str:
104     """
    ... 13 lines
118 @abstractmethod
119 async def launch_async(self, spec: ExecutionSpec) -> None:
120     """
    ... 7 lines
128 @abstractmethod
129 async def close_async(self) -> None:
130     """Release exactly the owned resources, collect artifacts, and stay idempotent."""
```

pyrit/agent/environment.py — 08-agent-framework · 383248e4a

— Contract · PyRIT — Check passed — Your code

Make PyRIT use my sandbox

EXECUTIONENVIRONMENTPROVIDER · THE FOUR MOMENTS

01 reserve()
Record deterministic resource identifiers before anything exists, so a crash mid-allocation stays reconcilable.

02 inference_endpoint_async(relay_url)
Translate the relay URL into your network namespace. Must not acquire resources.

03 launch_async(spec)
Seed the workspace, acquire resources, start the harness. Only variable names on command lines; values through your own channel.

04 close_async()
Release exactly what you own, collect artifacts, stay idempotent, refuse anything you did not create.

CONFORMANCEREPORT · PROVIDER 'LOCAL' · REAL RUN

17 / 17 CHECKS · ALL PASSED

- ✓ template names this provider
- ✓ provider is registered under its name
- ✓ unsandboxed providers require consent and isolated providers do not
- ✓ create_session returns an ExecutionSession
- ✓ acp_reader is unavailable before launch
- ✓ acp_writer is unavailable before launch
- ✓ stderr is unavailable before launch
- ✓ inference_endpoint_async translates at least one relay address and rejects others with ValueError
- ✓ launch_async starts the harness
- ✓ ACP byte streams are full-duplex and order-preserving
- ✓ closing ACP input ends the output stream
- ✓ stderr is readable to end of stream
- ✓ credential values never reach the record
- ✓ provenance is a string mapping
- ✓ close_async releases resources
- ✓ close_async is idempotent
- ✓ a repeated close_async leaves the record unchanged

REVIEW

A provider PR is reviewed against this report; Docker's runs in integration tests.

STEP 02 / 04

Run the conformance suite against a real allocation

`verify_environment_provider_async` drives one real allocation with a byte-echo command and reports one check per promise: streams unavailable before launch, relay translation, a secret that must not leak, idempotent release, honest capabilities. This report is the real one for the `local` provider, captured while building this deck: 17 of 17.

The seam in depth → Core owns policy, providers own resources

verify_environment_provider_async PR #17

```
129 async def verify_environment_provider_async(
    ⋯ 7 lines
137     """
138     Drive one real allocation through the provider contract with a byte-echo command.
139
140     ``echo_command`` must copy stdin to stdout inside the environment (for example ``("c
141     in a POSIX image). Nothing here speaks ACP; the checks cover resources, streams, sec
142     idempotent release, and capability honesty.
    ⋯ 1 lines
144     Returns:
145         ConformanceReport: One check per promise.
146     """
    ⋯ 2 lines
```

pyrit/agent/conformance.py 08-agent-framework · 383248e4a

— Contract · PyRIT — Check passed — Your code

Make PyRIT use my sandbox

EXECUTIONENVIRONMENTPROVIDER · THE FOUR MOMENTS

01 reserve()
Record deterministic resource identifiers before anything exists, so a crash mid-allocation stays reconcilable.

02 inference_endpoint_async(relay_url)
Translate the relay URL into your network namespace. Must not acquire resources.

03 launch_async(spec)
Seed the workspace, acquire resources, start the harness. Only variable names on command lines; values through your own channel.

04 close_async()
Release exactly what you own, collect artifacts, stay idempotent, refuse anything you did not create.

CONFORMANCEREPORT · PROVIDER 'LOCAL' · REAL RUN — 17 / 17 CHECKS · ALL PASSED

- ✓ template names this provider
- ✓ provider is registered under its name
- ✓ unsandboxed providers require consent and isolated providers do not
- ✓ create_session returns an ExecutionSession
- ✓ acp_reader is unavailable before launch
- ✓ acp_writer is unavailable before launch
- ✓ stderr is unavailable before launch
- ✓ inference_endpoint_async translates at least one relay address and rejects others with ValueError
- ✓ launch_async starts the harness
- ✓ ACP byte streams are full-duplex and order-preserving
- ✓ closing ACP input ends the output stream
- ✓ stderr is readable to end of stream
- ✓ credential values never reach the record
- ✓ provenance is a string mapping
- ✓ close_async releases resources
- ✓ close_async is idempotent
- ✓ a repeated close_async leaves the record unchanged

ACA

The Azure Container Apps sandbox provider was built this way outside the repository with zero core changes; packaging waits on the preview CLI.

STEP 03 / 04

Register through setup, never from the UI

Providers and adapters are trusted code registered by setup (`EnvironmentProviderRegistry.register`), not import strings accepted over REST. Templates then name your provider by its `info.name`, and the UI lists the template fields your provider declares.

EnvironmentProviderRegistry.register — PR #17

```
204 def register(self, provider: ExecutionEnvironmentProvider) -> None:
205     """
206     Install a trusted provider before creating templates/targets that refer to it.
207
208     Raises:
209         ValueError: A provider already owns the requested name.
210         TypeError: The implementation does not implement the provider contract.
211     """
212     if not isinstance(provider, ExecutionEnvironmentProvider):
213         raise TypeError("Environment providers must implement ExecutionEnvironmentProvic
214     name = provider.info.name
215     with self._lock:
```

pyrit/agent/environment.py — 08-agent-framework · 383248e4a

— Contract · PyRIT — Check passed — Your code

Make PyRIT use my sandbox

EXECUTIONENVIRONMENTPROVIDER · THE FOUR MOMENTS

01 reserve()
Record deterministic resource identifiers before anything exists, so a crash mid-allocation stays reconcilable.

02 inference_endpoint_async(relay_url)
Translate the relay URL into your network namespace. Must not acquire resources.

03 launch_async(spec)
Seed the workspace, acquire resources, start the harness. Only variable names on command lines; values through your own channel.

04 close_async()
Release exactly what you own, collect artifacts, stay idempotent, refuse anything you did not create.

CONFORMANCEREPORT · PROVIDER 'LOCAL' · REAL RUN — 17 / 17 CHECKS · ALL PASSED

- ✓ template names this provider
- ✓ provider is registered under its name
- ✓ unsandboxed providers require consent and isolated providers do not
- ✓ create_session returns an ExecutionSession
- ✓ acp_reader is unavailable before launch
- ✓ acp_writer is unavailable before launch
- ✓ stderr is unavailable before launch
- ✓ inference_endpoint_async translates at least one relay address and rejects others with ValueError
- ✓ launch_async starts the harness
- ✓ ACP byte streams are full-duplex and order-preserving
- ✓ closing ACP input ends the output stream
- ✓ stderr is readable to end of stream
- ✓ credential values never reach the record
- ✓ provenance is a string mapping
- ✓ close_async releases resources
- ✓ close_async is idempotent
- ✓ a repeated close_async leaves the record unchanged

RESERVED

Process identity and lookup paths (HOME, PATH, ...) can never be credential references.

STEP 04 / 04

A new harness is an adapter, not a fork

`HarnessAdapter` declares `info` (name, default command, wire APIs, native credential names, reserved variables), `launch_environment` (how the harness finds its private home) and `inference_environment` (how it is pointed at the relay). `verify_harness_adapter` checks those promises without running anything. Copilot CLI and the Agent Framework host are the two shipped adapters; a generic ACP adapter covers the rest.

Harnesses in depth → Copilot CLI, Agent Framework, any ACP agent

HarnessAdapter — PR #17

```
16  class HarnessAdapter(ABC):
    ··· 2 lines
19      # Process identity and lookup paths belong to the execution, never to credential ref
20      EXECUTION_RESERVED_ENVIRONMENT: ClassVar[frozenset[str]] = frozenset(
21          {
22              "HOME",
23              "USERPROFILE",
    ··· 4 lines
28              "LOCALAPPDATA",
29              "PATH",
30              "PATHEXT",
    ··· 5 lines
```

pyrit/agent/harness.py — 08-agent-framework · 383248e4a

— Contract · PyRIT — Check passed — Your code

LIMITS

Known limits

Said plainly, so reviewers do not have to find them.

Budgets are per execution
Default 100 inference requests, max 1,000; per-turn budgets are deferred.

Modality is declared, not policed per request
The binding declares input modalities; undeclared image parts are not yet rejected by the relay.

`local` is not a sandbox
Private home and workspace only; it requires an explicit acknowledgement.

Permission policy is not enforcement
It is a deterministic ACP answer; isolation comes from the environment.

Tool coverage is what the protocol exposes
Missing tool calls are undetermined, not false; the UI says so under every turn.

LIMITS

Deferred on purpose

Each has a place in the stack where it would land; none is needed to test an agent today.

Agent configuration registry UI · create by preset name
After 6b. Does **not** replace the three-panel form.

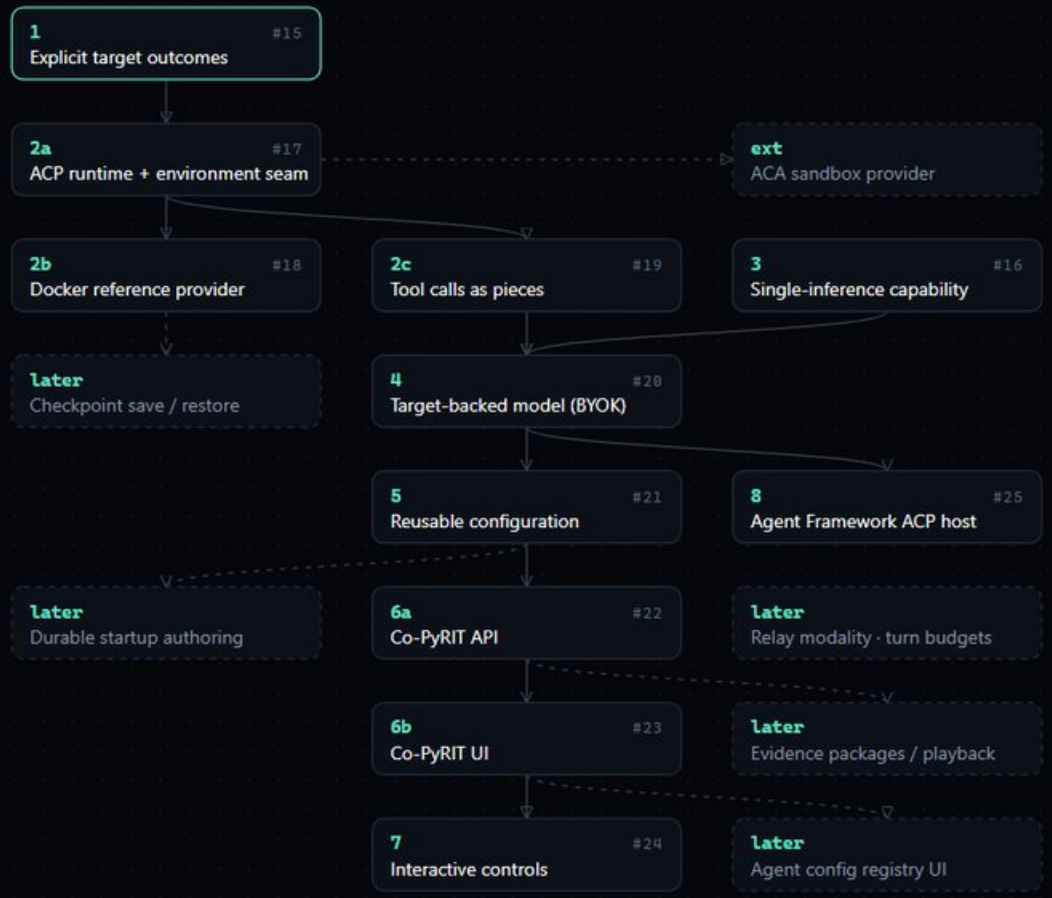
Checkpoint save / restore
After 2b, as an optional Docker-provider capability.

Durable startup authoring · evidence playback
After 5 / after 6a. "Download journal" covers today's need.

Relay modality enforcement · per-turn budgets
After 4; both change evidence semantics.

Managed cloud providers (ACA)
Proven out of tree; packaging waits on the preview CLI/SDK.

Eleven claims, one linear chain



HOVER OR FOCUS A NODE

#15 · AGENTS/STACK-01-OUTCOMES

Explicit target outcomes

A target can complete without assistant text. Every `TargetResponse` declares an explicit outcome instead of letting an empty string mean anything.

WHY SEPARATE	Core-model change touching every target; approved on its own merits.
NEEDS	—
REVIEWER	target maintainers

spencrr/PyRIT/pull/15

Promotion. #15 and #16 are independent and go first; each later PR is rebased onto `main` once its predecessor merges. Base: upstream `microsoft/PyRIT` main at `6a2319644`. The unsliced proof of concept lives on `agents/integration` (reference only). Story gist: gist.github.com/spencrr/474c66a5f7c569106b5068b7477d4e90

EPILOGUE

How to review

Each PR carries its feature commit plus the fixes from two skeptical reviews as separate FIX/TEST commits, so you can see what the review changed. Each PR body lists the exact commands run and their results.

Start with #15 and #16
Independent core/target changes; natural first promotions to `microsoft/PyRIT`.

Read 2a against the conformance report
Run `verify_environment_provider_async` for `local`; the Docker report is in 2b's tests.

Give 2c undivided attention
The projection rules and omission reasons are the false-evidence boundary.

Treat 4 as a trust boundary
Lease, body validation, reserved variables, redaction of the token.

Validation on top of the stack
`pytest tests/unit -n 4` 30,904 passed · frontend 166 suites / 3,767 tests · ruff, tsc, eslint, `uv lock --check` · 7 agent integration tests against a real scripted ACP process and the Agent Framework host.

EPILOGUE

Questions, objections, better cuts

Drafts #15–#25 on `spencrr/PyRIT`, GitHub stack #26. The story gist links every PR, the four diagrams, and this deck. Nothing is promoted to `microsoft/PyRIT` yet; that starts when you say so.

Two tracks

Tour

(this): pitch, swappable, map, three perspectives, stack.

Deep dive

(press T): one turn, seam, journal → pieces, relay, identity, approvals, harnesses, decisions.

Real throughout

Journals, projection and conformance report captured from the scripted ACP agent; Co-PyRIT screenshots captured headlessly on this branch; every code panel pinned to a branch tip.

Keys

← → steps · Tab chapter · T track · Space autoplay · E evidence · P presenter · Esc overview · hover any dotted `term` for its definition